

P8106 Data Science II — Final Project (Code & Output)

Yongyan Liu (yl6107)

2026-05-09

Contents

1. Setup, data, and train/test split	2
2. Exploratory data analysis	3
3. Model training	4
3.1 Logistic regression (GLM)	4
3.2 Penalized logistic regression (elastic net, GLMNET)	4
3.3 GAM	5
3.4 MARS	6
3.5 LDA	6
3.6 Random forest	6
3.7 Boosting (AdaBoost via gbm)	7
3.8 SVM (linear kernel)	8
3.9 SVM (radial-basis kernel)	9
4. Model comparison	10
4.1 Cross-validated ROC (resamples)	10
4.2 Test-set metrics	12
5. Interpretation of the best model	12
5.1 Variable importance (boosting)	13
5.2 Partial dependence plots — top 4 predictors	14
5.3 ROC curves on the test set and bootstrap AUC CIs	15
5.4 Calibration of the best-model risk score	17
6. Export results for the report	17

This file contains all R code that produces the numbers, tables, and figures presented in the analysis report `p8106_final_yl6107_report.pdf`.

1. Setup, data, and train/test split

```
library(tidyverse)
library(caret)
library(pROC)
library(pdp)
library(gridExtra)
library(MASS)
library(mgcv)
library(earth)
library(glmnet)
library(ranger)
library(gbm)
library(e1071)
library(kernlab)
## Optional: parallelise caret's CV (multi-fold fits run on separate cores).
## Comment out the next 3 lines if doParallel is not installed.
library(doParallel)
cl <- makeCluster(max(1, parallel::detectCores() - 1))
registerDoParallel(cl)
```

Sample 1,000 observations from the full dataset using the last four digits of the UNI as the random seed, then split exactly 800 / 200.

```
load("severity.RData") # loads `dat`

set.seed(6107)
my_dat <- dat[sample(nrow(dat), 1000), ]
my_dat$id <- NULL

my_dat <- my_dat |>
  mutate(
    gender      = factor(gender,      levels = c(0, 1), labels = c("Female", "Male")),
    diabetes    = factor(diabetes,    levels = c(0, 1), labels = c("No", "Yes")),
    hypertension = factor(hypertension, levels = c(0, 1), labels = c("No", "Yes")),
    vaccine     = factor(vaccine,     levels = c(0, 1), labels = c("No", "Yes")),
    race        = factor(race, levels = c(1, 2, 3, 4),
                        labels = c("White", "Asian", "Black", "Hispanic")),
    smoking     = factor(smoking, levels = c(0, 1, 2),
                        labels = c("Never", "Former", "Current")),
    severity    = factor(severity, levels = c(1, 0), labels = c("Severe", "NotSev"))
  )

set.seed(6107)
train_idx <- sample(nrow(my_dat), 800)
train_d   <- my_dat[train_idx, ]
test_d    <- my_dat[-train_idx, ]
```

Shared 10-fold CV indices so that the resamples comparison across all models uses the same folds.

```
set.seed(6107)
cvIndex <- createFolds(train_d$severity, k = 10, returnTrain = TRUE)
ctrl <- trainControl(method = "cv",
                     number = 10,
                     index = cvIndex,
                     classProbs = TRUE,
                     summaryFunction = twoClassSummary)
```

2. Exploratory data analysis

```
summary_tab <- train_d |>
  pivot_longer(c(age, height, weight, bmi, SBP, LDL, depression),
              names_to = "Var", values_to = "v") |>
  group_by(severity, Var) |>
  summarise(`Mean (SD)` = sprintf("%.1f (%.1f)", mean(v), sd(v)),
            .groups = "drop") |>
  pivot_wider(names_from = severity, values_from = `Mean (SD)`)
summary_tab
```

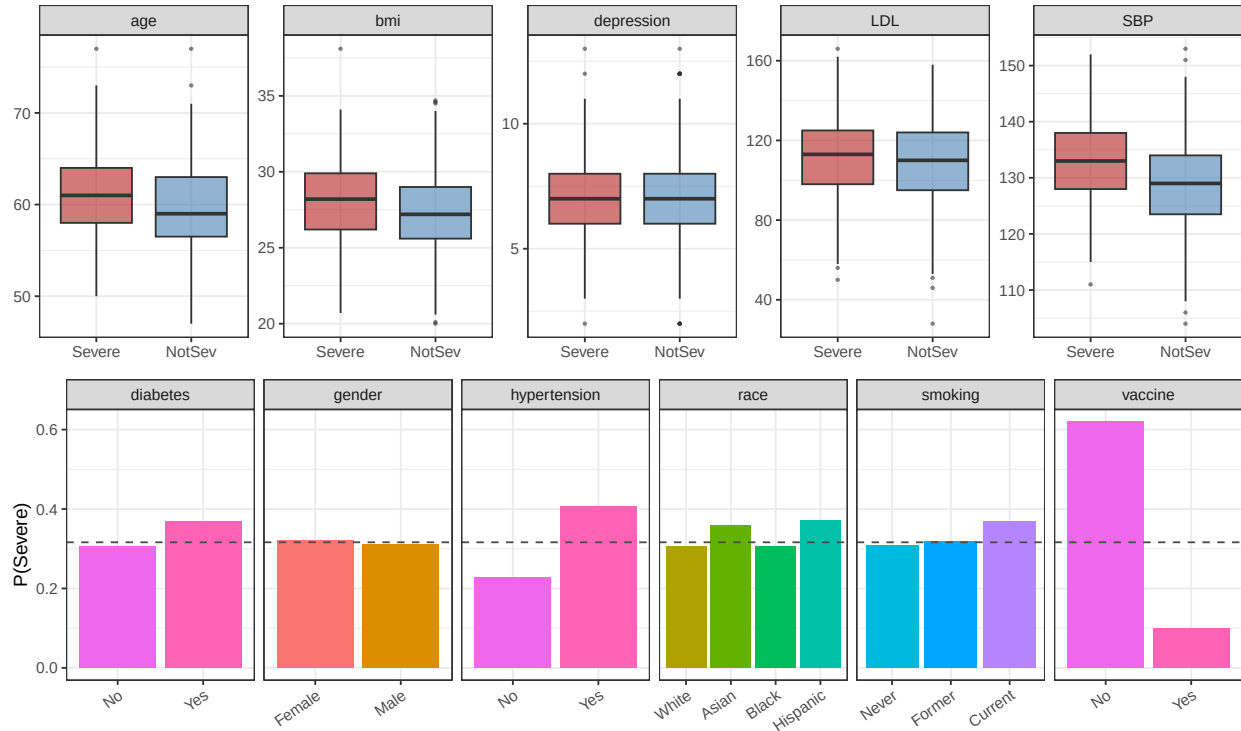
```
## # A tibble: 7 x 3
##   Var      Severe      NotSev
##   <chr>    <chr>      <chr>
## 1 LDL      111.9 (20.3) 109.3 (20.0)
## 2 SBP      132.8 (7.5) 129.0 (7.9)
## 3 age       61.3 (4.4)  59.6 (4.4)
## 4 bmi       28.0 (2.7) 27.3 (2.6)
## 5 depression 7.1 (1.9)  7.1 (2.0)
## 6 height    169.7 (6.4) 170.6 (5.9)
## 7 weight    80.5 (6.8)  79.3 (6.9)
```

```
num_vars <- c("age", "bmi", "SBP", "LDL", "depression")
p_num <- train_d |>
  pivot_longer(all_of(num_vars), names_to = "Var", values_to = "v") |>
  ggplot(aes(severity, v, fill = severity)) +
  geom_boxplot(alpha = 0.6, outlier.size = 0.5) +
  facet_wrap(~ Var, scales = "free_y", nrow = 1) +
  scale_fill_manual(values = c("Severe" = "firebrick", "NotSev" = "steelblue")) +
  labs(x = NULL, y = NULL) +
  theme_bw() + theme(legend.position = "none")

cat_vars <- c("gender", "race", "smoking", "diabetes", "hypertension", "vaccine")
p_cat <- train_d |>
  pivot_longer(all_of(cat_vars), names_to = "Var", values_to = "lvl") |>
  group_by(Var, lvl) |>
  summarise(rate = mean(severity == "Severe"), n = n(), .groups = "drop") |>
  ggplot(aes(lvl, rate, fill = lvl)) +
  geom_col() + geom_hline(yintercept = mean(train_d$severity == "Severe"),
                        linetype = "dashed", colour = "grey30") +
  facet_wrap(~ Var, scales = "free_x", nrow = 1) +
  labs(x = NULL, y = "P(Severe)") +
```

```
theme_bw() + theme(legend.position = "none",
  axis.text.x = element_text(angle = 35, hjust = 1))

grid.arrange(p_num, p_cat, nrow = 2)
```



3. Model training

All tuning grids were extended (when needed) until the CV-selected hyperparameter was strictly interior — i.e. not on a grid boundary.

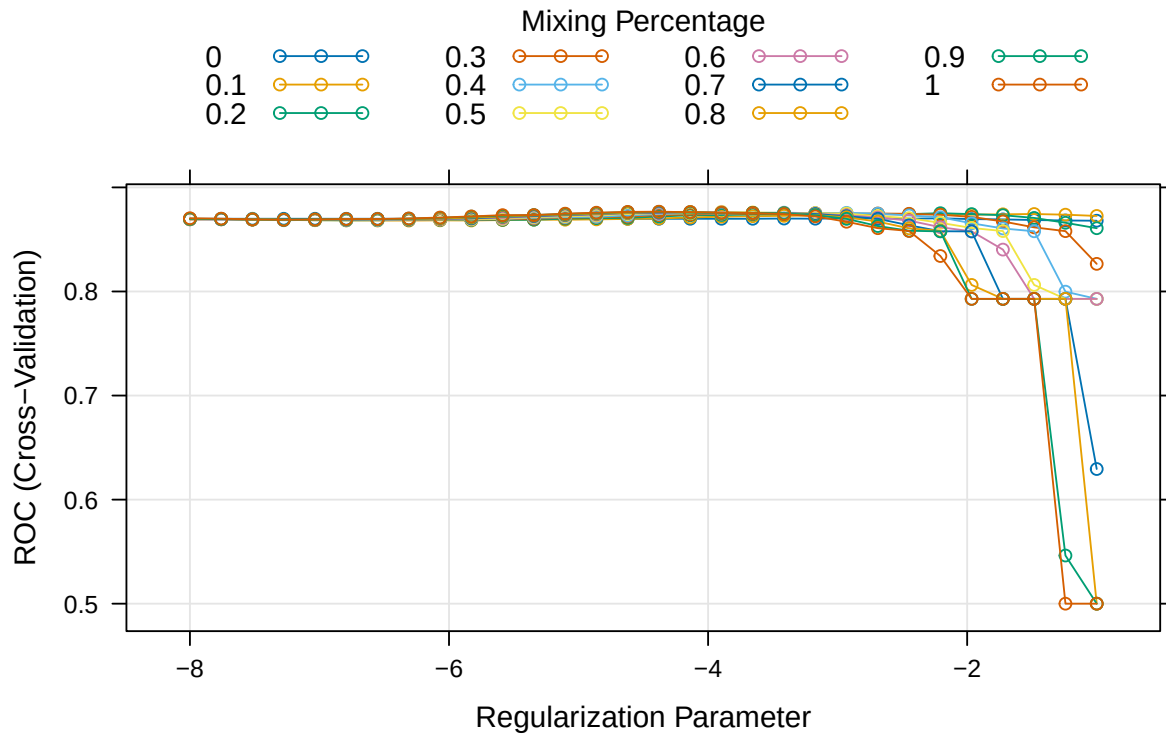
3.1 Logistic regression (GLM)

```
set.seed(6107)
fit.glm <- train(severity ~ ., data = train_d,
  method = "glm", metric = "ROC", trControl = ctrl)
```

3.2 Penalized logistic regression (elastic net, GLMNET)

```
glmnetGrid <- expand.grid(alpha = seq(0, 1, length = 11),
  lambda = exp(seq(-8, -1, length = 30)))
set.seed(6107)
fit.glmnet <- train(severity ~ ., data = train_d,
  method = "glmnet", tuneGrid = glmnetGrid,
```

```
metric = "ROC", trControl = ctrl)
plot(fit.glmn, xTrans = log)
```



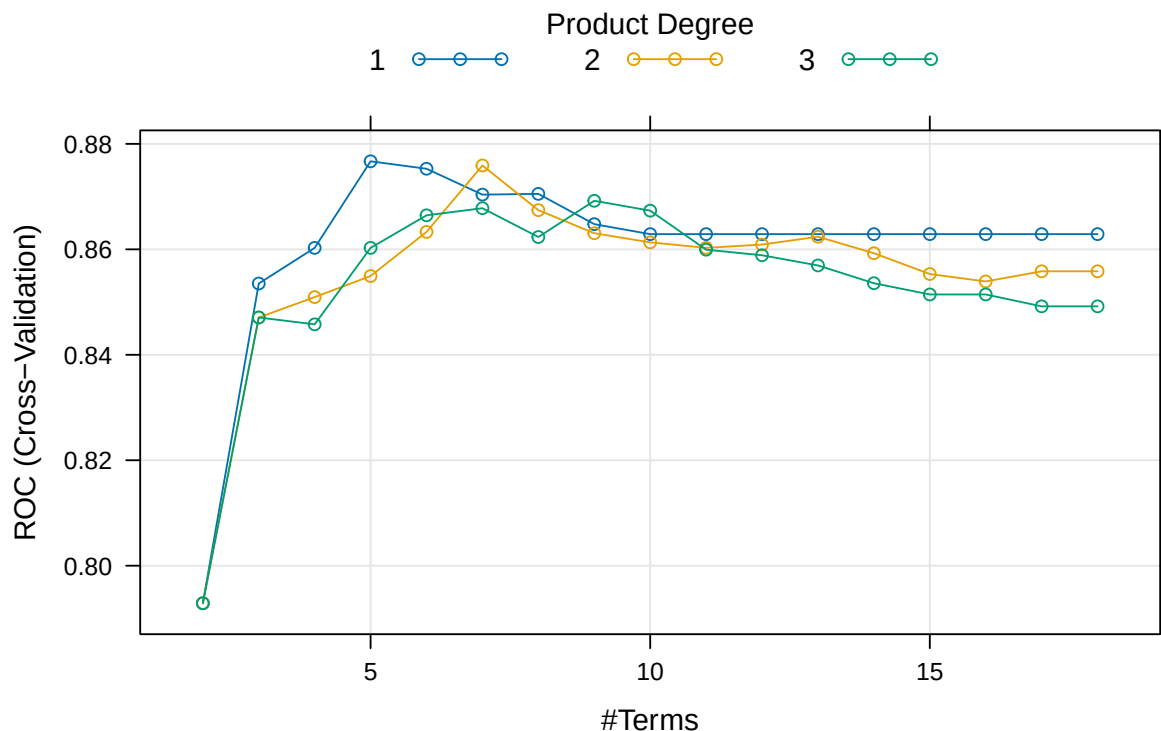
3.3 GAM

```
set.seed(6107)
fit.gam <- train(severity ~ ., data = train_d,
                 method = "gam", metric = "ROC", trControl = ctrl)
fit.gam$finalModel

##
## Family: binomial
## Link function: logit
##
## Formula:
## .outcome ~ genderMale + raceAsian + raceBlack + raceHispanic +
##   smokingFormer + smokingCurrent + diabetesYes + hypertensionYes +
##   vaccineYes + s(depression) + s(age) + s(SBP) + s(LDL) + s(bmi) +
##   s(height) + s(weight)
##
## Estimated degrees of freedom:
## 1.83 2.58 1.00 1.00 1.00 1.00 1.00
## total = 19.41
##
## UBRE score: -0.1818651
```

3.4 MARS

```
marsGrid <- expand.grid(degree = 1:3, nprune = 2:18)
set.seed(6107)
fit.mars <- train(severity ~ ., data = train_d,
                  method = "earth", tuneGrid = marsGrid,
                  metric = "ROC", trControl = ctrl)
plot(fit.mars)
```



3.5 LDA

```
set.seed(6107)
fit.lda <- train(severity ~ ., data = train_d,
                 method = "lda", metric = "ROC", trControl = ctrl)
```

3.6 Random forest

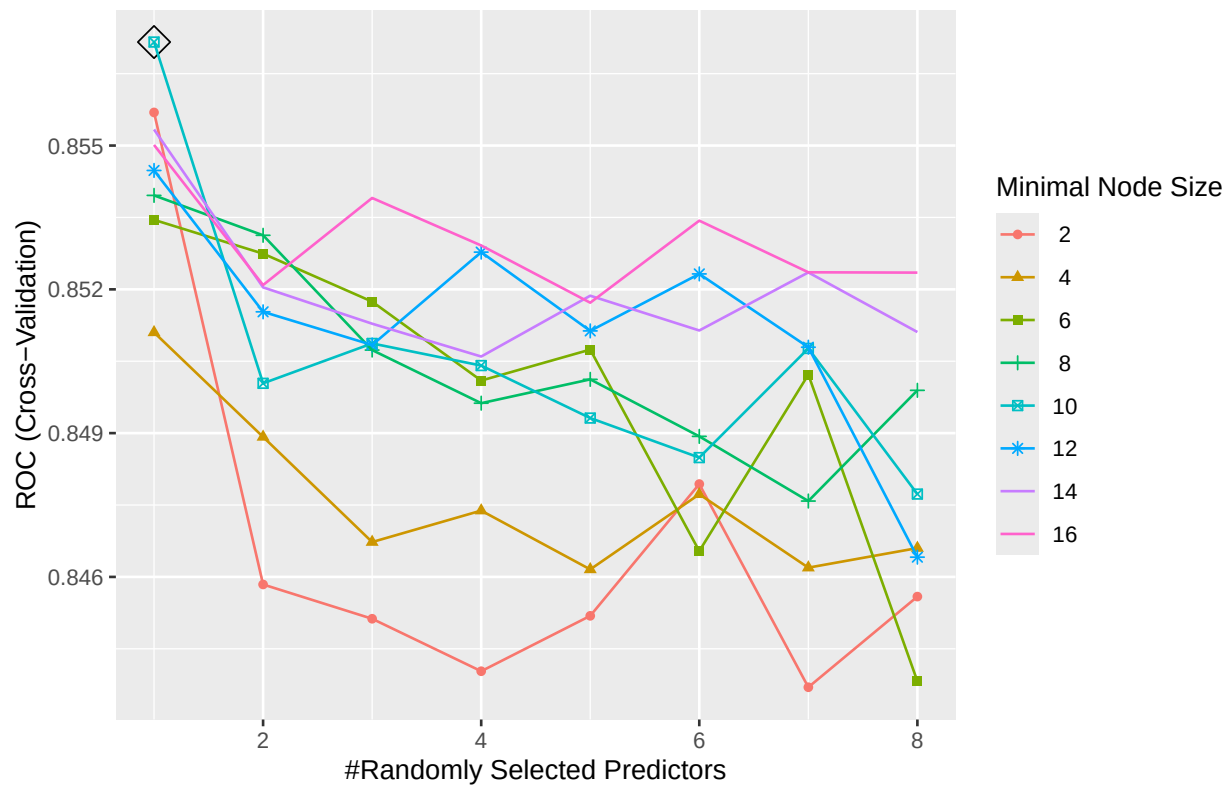
mtry swept from 1 to 8 and min.node.size over {2, 4, ..., 16} (L11 lecture template). Beyond min.node.size 80 the trees become near-stubs and CV ROC drifts upward in a degenerate manner unrelated to predictive ability, so we cap the grid at 16.

```
rf.grid <- expand.grid(mtry = 1:8,
                      splitrule = "gini",
```

```

min.node.size = seq(2, 16, by = 2))
set.seed(6107)
fit.rf <- train(severity ~ ., data = train_d,
               method = "ranger", tuneGrid = rf.grid,
               metric = "ROC", trControl = ctrl,
               importance = "permutation")
ggplot(fit.rf, highlight = TRUE)

```



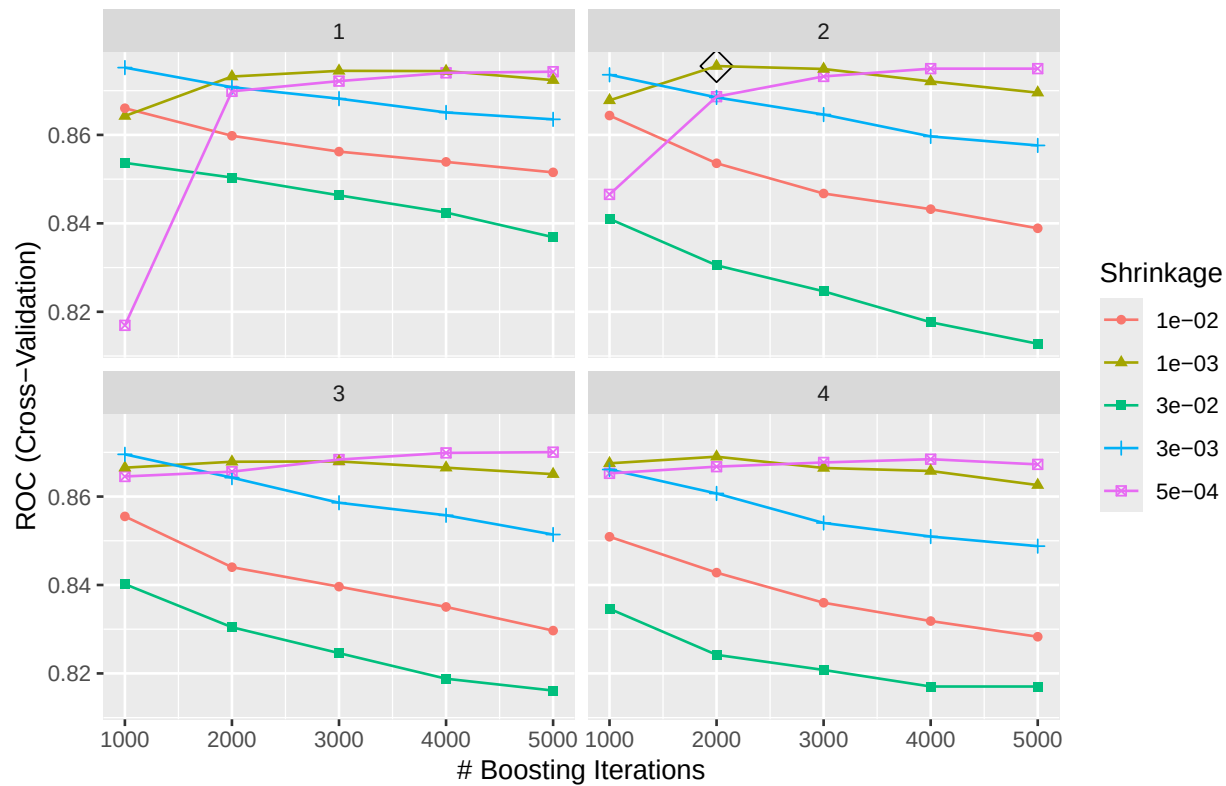
3.7 Boosting (AdaBoost via gbm)

shrinkage swept over four orders of magnitude (0.0005 to 0.03) with `n.trees` up to 5,000 so that the CV-selected triple (`n.trees`, `interaction.depth`, `shrinkage`) sits in the interior.

```

gbmA.grid <- expand.grid(n.trees      = c(1000, 2000, 3000, 4000, 5000),
                       interaction.depth = 1:4,
                       shrinkage      = c(0.0005, 0.001, 0.003, 0.01, 0.03),
                       n.minobsinnode = 10)
set.seed(6107)
fit.gbmA <- train(severity ~ ., data = train_d,
                 method = "gbm", tuneGrid = gbmA.grid,
                 metric = "ROC", trControl = ctrl,
                 distribution = "adaboost", verbose = FALSE)
ggplot(fit.gbmA, highlight = TRUE)

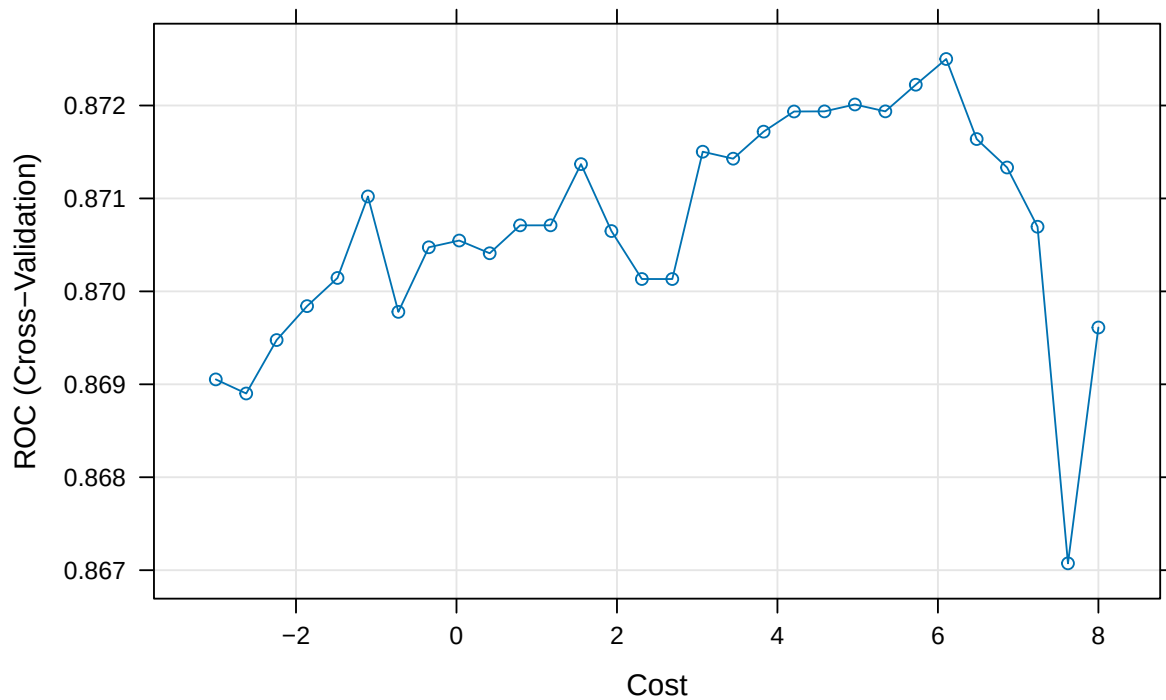
```



3.8 SVM (linear kernel)

Cost grid extended to e^8 to push the CV-selected C strictly interior.

```
svml.grid <- data.frame(C = exp(seq(-3, 8, length = 30)))
set.seed(6107)
fit.svml <- train(severity ~ ., data = train_d,
  method = "svmLinear", tuneGrid = svml.grid,
  metric = "ROC", trControl = ctrl,
  preProcess = c("center", "scale"))
plot(fit.svml, xTrans = log, highlight = TRUE)
```

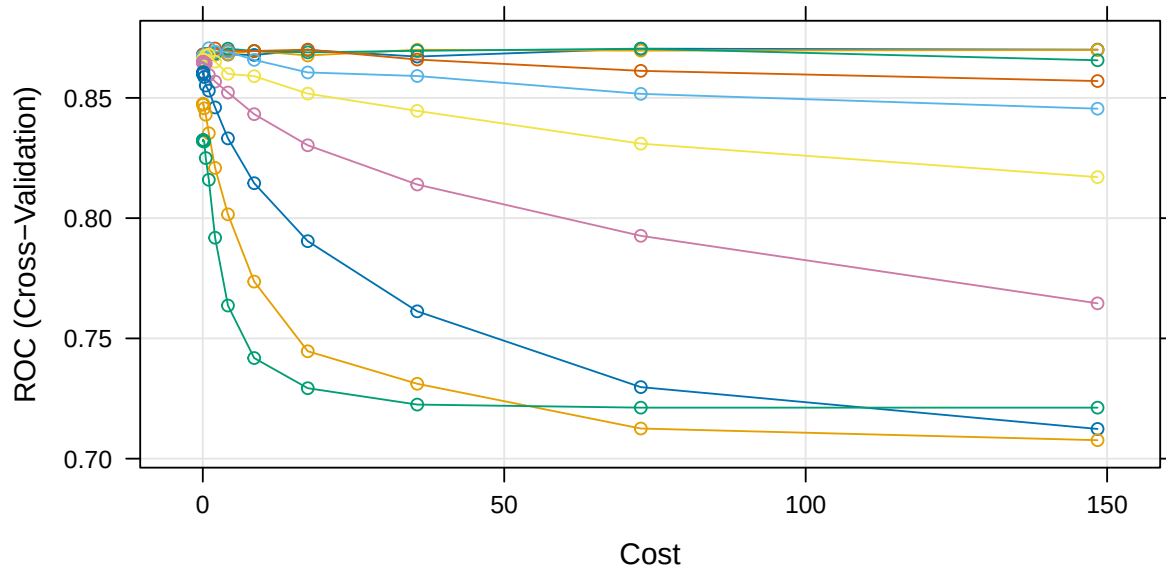



3.9 SVM (radial-basis kernel)

Cost grid extended down to e^{-5} so the CV-selected C sits in the interior.

```
svmr.grid <- expand.grid(C      = exp(seq(-5, 5, length = 15)),
                        sigma = exp(seq(-8, -2, length = 10)))
set.seed(6107)
fit.svmr <- train(severity ~ ., data = train_d,
                  method = "svmRadialSigma", tuneGrid = svmr.grid,
                  metric = "ROC", trControl = ctrl,
                  preProcess = c("center", "scale"))
plot(fit.svmr, highlight = TRUE)
```

			Sigma		
12		0.00247875217666636		0.0183156388887342	
81		0.00482794999383144		0.0356739933472524	
11		0.00940356255149521		0.0694834512228015	



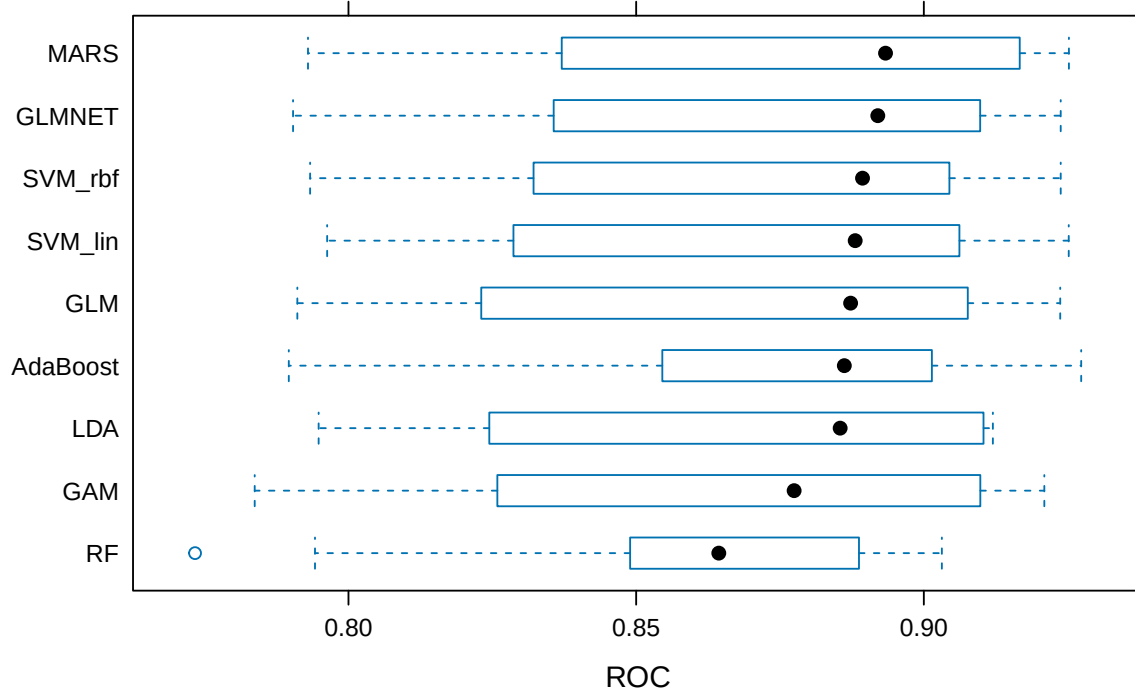
4. Model comparison

4.1 Cross-validated ROC (resamples)

```
resamp <- resamples(list(GLM = fit.glm, GLMNET = fit.glmn,
                        GAM = fit.gam, MARS = fit.mars,
                        LDA = fit.lda, RF = fit.rf,
                        AdaBoost = fit.gbmA,
                        SVM_lin = fit.svml, SVM_rbf = fit.svmr))
summary(resamp)$statistics$ROC
```

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
## GLM	0.7911111	0.8353077	0.8872727	0.8706208	0.9032797	0.9237037	0
## GLMNET	0.7903704	0.8487483	0.8920000	0.8766537	0.9063427	0.9237892	0
## GAM	0.7837037	0.8374056	0.8774545	0.8675111	0.9050909	0.9209402	0
## MARS	0.7929630	0.8456154	0.8933427	0.8766955	0.9140455	0.9252137	0
## LDA	0.7948148	0.8378112	0.8854545	0.8702153	0.9072533	0.9120000	0
## RF	0.7733333	0.8492587	0.8643636	0.8571598	0.8872862	0.9031339	0
## AdaBoost	0.7896296	0.8592424	0.8861818	0.8755712	0.9007762	0.9273504	0
## SVM_lin	0.7962963	0.8391399	0.8880699	0.8725006	0.9040000	0.9251852	0
## SVM_rbf	0.7933333	0.8403077	0.8893427	0.8706602	0.9026970	0.9237892	0

```
bwplot(resamp, metric = "ROC")
```



Pairwise Bonferroni-corrected paired tests on the shared CV folds — used in the report to back the “no” answer to the boosting/SVM question.

```
diff_resamp <- diff(resamp, metric = "ROC")
summary(diff_resamp)
```

```
##
## Call:
## summary.diff.resamples(object = diff_resamp)
##
## p-value adjustment: bonferroni
## Upper diagonal: estimates of the difference
## Lower diagonal: p-value for H0: difference = 0
##
## ROC
##      GLM      GLMNET      GAM      MARS      LDA      RF
## GLM              -6.033e-03  3.110e-03 -6.075e-03  4.055e-04  1.346e-02
## GLMNET  1.0000              9.143e-03 -4.185e-05  6.438e-03  1.949e-02
## GAM    1.0000 0.5903              -9.184e-03 -2.704e-03  1.035e-02
## MARS   1.0000 1.0000      0.8971              6.480e-03  1.954e-02
## LDA    1.0000 1.0000      1.0000      1.0000              1.306e-02
## RF     1.0000 0.1480      1.0000      0.4305      1.0000
## AdaBoost 1.0000 1.0000      1.0000      1.0000      1.0000      0.1232
## SVM_lin  1.0000 1.0000      1.0000      1.0000      1.0000      1.0000
## SVM_rbf  1.0000 1.0000      1.0000      1.0000      1.0000      0.8687
##
##      AdaBoost      SVM_lin      SVM_rbf
## GLM    -4.950e-03 -1.880e-03 -3.942e-05
## GLMNET  1.082e-03  4.153e-03  5.993e-03
```

```
## GAM      -8.060e-03 -4.990e-03 -3.149e-03
## MARS      1.124e-03  4.195e-03  6.035e-03
## LDA      -5.356e-03 -2.285e-03 -4.449e-04
## RF       -1.841e-02 -1.534e-02 -1.350e-02
## AdaBoost          3.071e-03  4.911e-03
## SVM_lin  1.0000          1.840e-03
## SVM_rbf  1.0000      1.0000
```

4.2 Test-set metrics

```
fits <- list(GLM = fit.glm, GLMNET = fit.glmn, GAM = fit.gam,
             MARS = fit.mars, LDA = fit.lda, RF = fit.rf,
             AdaBoost = fit.gbmA, SVM_lin = fit.svml, SVM_rbf = fit.svmr)

test_perf <- map_dfr(names(fits), function(nm) {
  m <- fits[[nm]]
  pp <- predict(m, newdata = test_d, type = "prob")[, "Severe"]
  pc <- predict(m, newdata = test_d)
  r <- roc(test_d$severity, pp, levels = c("NotSev", "Severe"))
  cm <- confusionMatrix(pc, test_d$severity, positive = "Severe")
  tibble(Model = nm,
          AUC = as.numeric(r$auc),
          Accuracy = as.numeric(cm$overall["Accuracy"]),
          Sens = as.numeric(cm$byClass["Sensitivity"]),
          Spec = as.numeric(cm$byClass["Specificity"]),
          Brier = mean((pp - (test_d$severity == "Severe"))^2))
})
test_perf %>% arrange(desc(AUC))
```

```
## # A tibble: 9 x 6
##   Model      AUC Accuracy  Sens  Spec Brier
##   <chr>    <dbl>    <dbl> <dbl> <dbl> <dbl>
## 1 RF      0.913    0.775 0.259 0.986 0.150
## 2 GAM      0.910    0.85  0.724 0.901 0.106
## 3 GLM      0.909    0.85  0.724 0.901 0.107
## 4 GLMNET   0.909    0.86  0.759 0.901 0.107
## 5 LDA      0.909    0.845 0.776 0.873 0.108
## 6 SVM_lin  0.907    0.855 0.724 0.908 0.108
## 7 SVM_rbf  0.907    0.835 0.741 0.873 0.109
## 8 AdaBoost 0.906    0.86  0.672 0.937 0.114
## 9 MARS     0.900    0.87  0.759 0.915 0.111
```

5. Interpretation of the best model

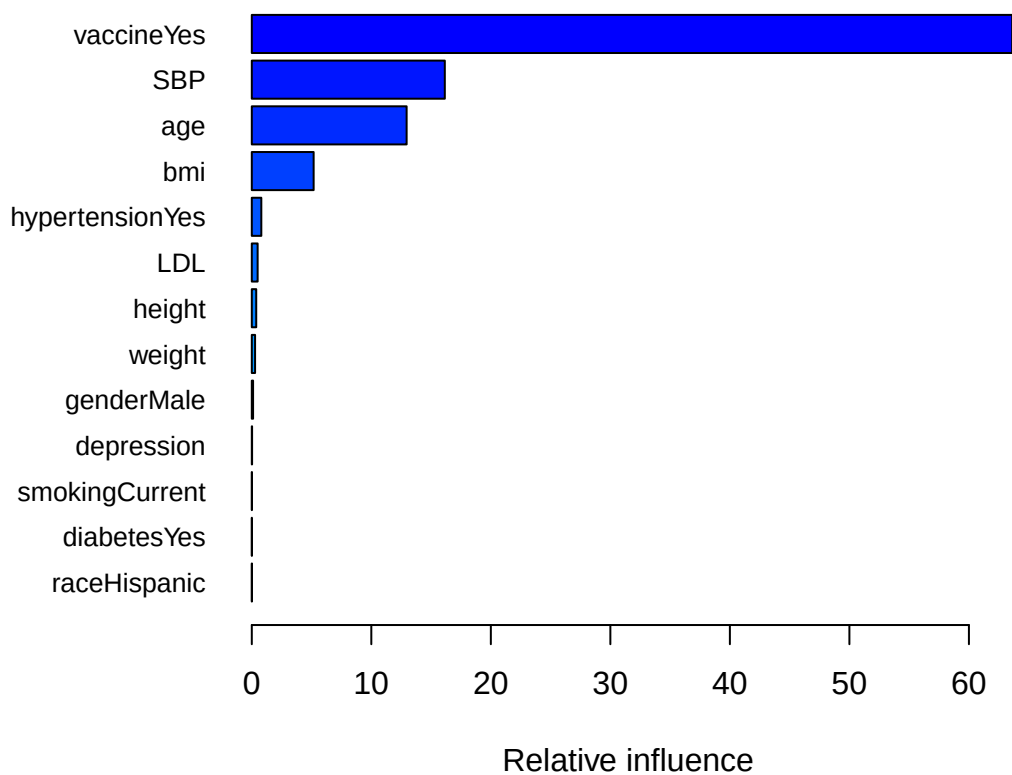
```
## Test AUCs are bunched within ~0.013 and bootstrap CIs overlap completely
## (see Section 4.2). We rank by 10-fold CV ROC, where MARS and GLMNET tie
## at the top. We pick MARS over GLMNET because (i) MARS is more parsimonious
## (4 predictors vs GLMNET's 7 non-zero linear terms at alpha=1, lambda=0.0125)
## and (ii) MARS hinge breakpoints read directly as clinical thresholds
```

```
## (effect is zero below the breakpoint, rises linearly above), whereas
## GLMNET keeps a linear effect over each variable's full range.
best_name <- "MARS"
best_fit <- fits[[best_name]]
best_name
```

```
## [1] "MARS"
```

5.1 Variable importance (boosting)

```
par(mar = c(4, 8, 2, 2))
vi_tab <- gbm::summary.gbm(fit.gbmA$finalModel, plotit = TRUE,
                           las = 1, cBars = 13, cex.names = 0.8)
```



```
vi_tab
```

```
##           var      rel.inf
## vaccineYes vaccineYes 63.620836202
## SBP          SBP    16.150149335
## age          age    12.954354208
## bmi          bmi     5.174496784
## hypertensionYes hypertensionYes 0.796856444
```

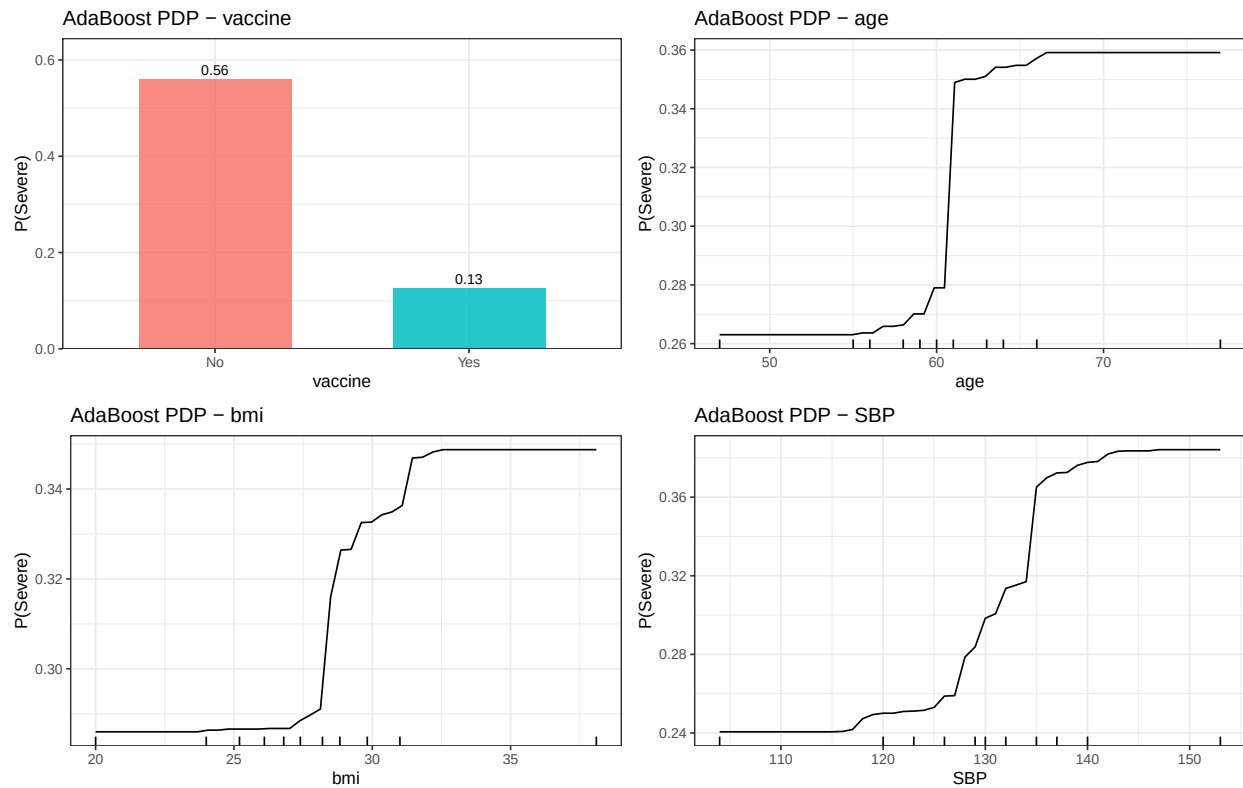
```
## LDL                LDL 0.488861717
## height             height 0.371031562
## weight             weight 0.275049458
## genderMale         genderMale 0.106986705
## depression         depression 0.025575659
## smokingCurrent     smokingCurrent 0.010910751
## diabetesYes        diabetesYes 0.009282511
## raceHispanic       raceHispanic 0.008958107
## smokingFormer      smokingFormer 0.006650557
## raceAsian          raceAsian 0.000000000
## raceBlack          raceBlack 0.000000000
```

5.2 Partial dependence plots — top 4 predictors

Compute PDP data for the four dominant predictors (vaccine, age, bmi, SBP). We store the data frames so the report can render the figure from `results.RData` without re-running `pdp::partial`.

```
pdp_vars <- c("vaccine", "age", "bmi", "SBP")
pdp_data <- lapply(pdp_vars, function(v) {
  pdp::partial(fit.gbma, pred.var = v, prob = TRUE,
               grid.resolution = 50, train = train_d)
})
names(pdp_data) <- pdp_vars

pdp_plot <- function(v) {
  df <- pdp_data[[v]]
  if (is.factor(df[[1]])) {
    ggplot(df, aes(.data[[v]], yhat, fill = .data[[v]])) +
      geom_col(width = 0.6, alpha = 0.85) +
      geom_text(aes(label = sprintf("%.2f", yhat)),
                vjust = -0.4, size = 3.2) +
      scale_y_continuous(expand = expansion(mult = c(0, 0.15))) +
      labs(title = paste("AdaBoost PDP -", v),
           x = v, y = "P(Severe)") +
      theme_bw() + theme(legend.position = "none")
  } else {
    autoplot(df, rug = TRUE, train = train_d) +
      labs(title = paste("AdaBoost PDP -", v),
           x = v, y = "P(Severe)") +
      theme_bw()
  }
}
do.call(grid.arrange, c(lapply(pdp_vars, pdp_plot), nrow = 2))
```



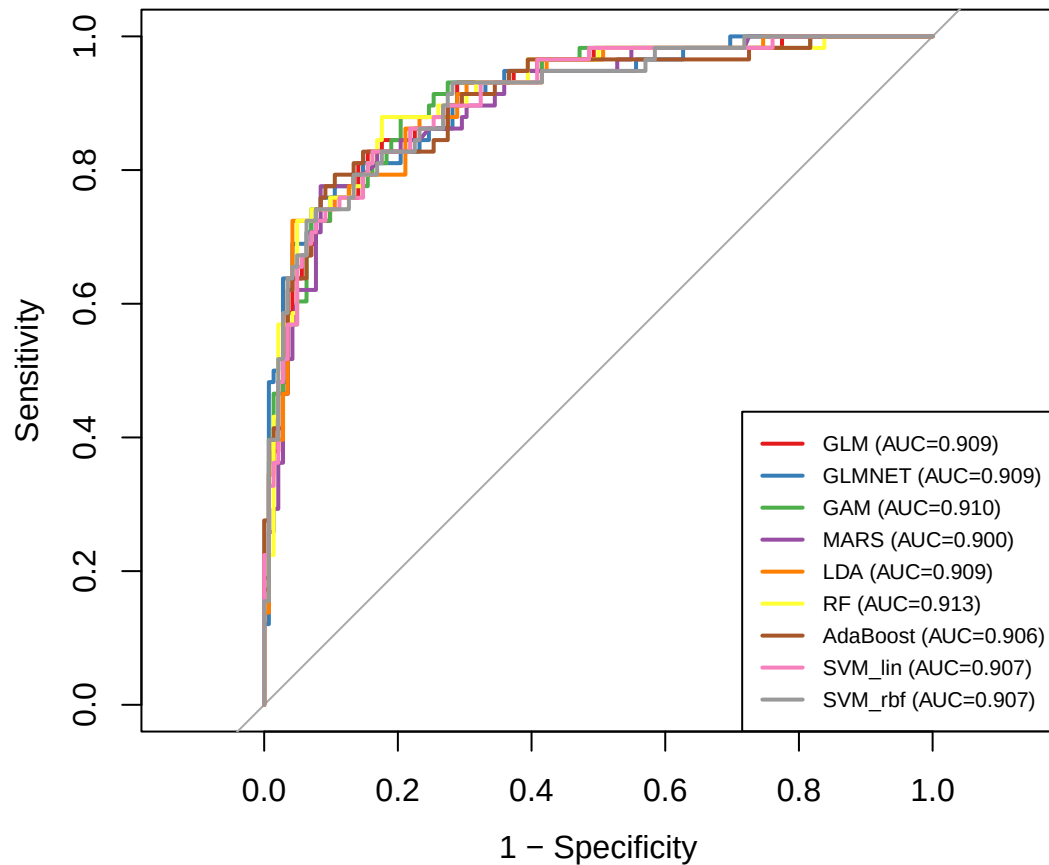
5.3 ROC curves on the test set and bootstrap AUC CIs

```

preds <- map(fits, ~ predict(.x, newdata = test_d, type = "prob")[, "Severe"])
rocs <- map(preds, ~ roc(test_d$severity, .x, levels = c("NotSev", "Severe")))

cols <- RColorBrewer::brewer.pal(9, "Set1")
plot(rocs[[1]], col = cols[1], legacy.axes = TRUE)
for (i in seq_along(rocs)[-1]) plot(rocs[[i]], add = TRUE, col = cols[i])
legend("bottomright",
      legend = sprintf("%s (AUC=%.3f)", names(rocs),
                        sapply(rocs, function(r) as.numeric(r$auc))),
      col = cols, lwd = 2, cex = 0.7)

```



2,000-replicate stratified bootstrap 95% CIs for the test-set AUC of each model (computed once here and reused by the report).

```
set.seed(6107)
auc_ci_df <- map_dfr(names(fits), function(nm) {
  ci <- pROC::ci.auc(rocs[[nm]], method = "bootstrap", boot.n = 2000)
  tibble(Model = nm,
    AUC_CI = sprintf("%.3f (%.3f, %.3f)",
      as.numeric(rocs[[nm]]$auc),
      as.numeric(ci)[1],
      as.numeric(ci)[3]))
})
auc_ci_df
```

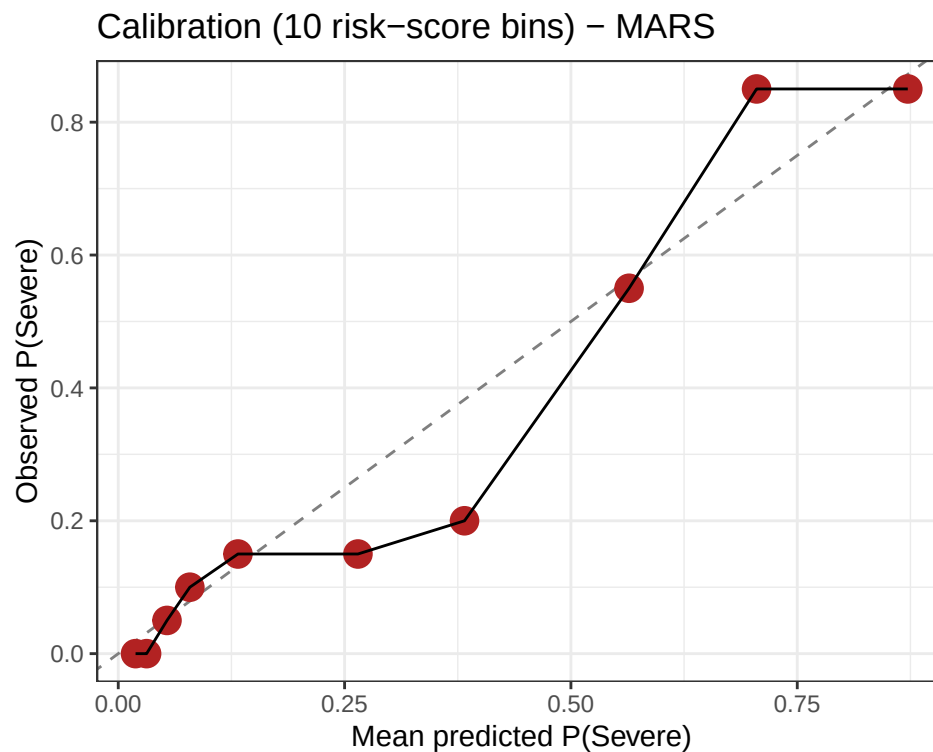
```
## # A tibble: 9 x 2
##   Model   AUC_CI
##   <chr>   <chr>
## 1 GLM     0.909 (0.863, 0.950)
## 2 GLMNET  0.909 (0.859, 0.950)
## 3 GAM     0.910 (0.862, 0.948)
## 4 MARS    0.900 (0.850, 0.945)
## 5 LDA     0.909 (0.860, 0.949)
## 6 RF      0.913 (0.866, 0.952)
```



```
## 7 AdaBoost 0.906 (0.854, 0.948)
## 8 SVM_lin 0.907 (0.859, 0.949)
## 9 SVM_rbf 0.907 (0.856, 0.949)
```

5.4 Calibration of the best-model risk score

```
pp_best <- predict(best_fit, newdata = test_d, type = "prob")[, "Severe"]
calib_df <- tibble(p = pp_best, y = as.integer(test_d$severity == "Severe")) |>
  mutate(bin = ntile(p, 10)) |>
  group_by(bin) |>
  summarise(p_mean = mean(p), obs = mean(y), n = n(), .groups = "drop")
ggplot(calib_df, aes(p_mean, obs)) +
  geom_abline(linetype = "dashed", colour = "grey50") +
  geom_point(aes(size = n), colour = "firebrick") +
  geom_line() +
  labs(x = "Mean predicted P(Severe)", y = "Observed P(Severe)",
       title = sprintf("Calibration (10 risk-score bins) - %s", best_name)) +
  theme_bw() + theme(legend.position = "none")
```



6. Export results for the report

The analysis report (`p8106_final_y16107_report.Rmd`) only **reads** from `results.RData` and renders — it does not refit models or recompute PDPs/bootstrap CIs. Knit this file first to regenerate `results.RData`, then knit the report.

```
save(my_dat, train_d, test_d, ctrl, cvIndex,  
     fits, resamp, test_perf,  
     best_name, best_fit,  
     vi_tab, pdp_data, rocs, calib_df, auc_ci_df,  
     file = "results.RData")  
try(stopCluster(cl), silent = TRUE)
```